



Playing with Our CAT and Communication-Centric Applications

Davide Basile, Pierpaolo Degano, Gian-Luigi Ferrari, Emilio Tuosto

► To cite this version:

Davide Basile, Pierpaolo Degano, Gian-Luigi Ferrari, Emilio Tuosto. Playing with Our CAT and Communication-Centric Applications. 36th International Conference on Formal Techniques for Distributed Objects, Components, and Systems (FORTE), Jun 2016, Heraklion, Greece. pp.62-73, <10.1007/978-3-319-39570-8_5>. <hal-01432928>

HAL Id: hal-01432928

<https://inria.hal.science/hal-01432928v1>

Submitted on 12 Jan 2017

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY 4.0 - Attribution - International License

Playing with our CAT and Communication-Centric Applications [★]

Davide Basile^{1 2}, Pierpaolo Degano², Gian-Luigi Ferrari², and Emilio Tuosto³

dbasile@isti.cnr.it {degano, giangi}@di.unipi.it emilio@le.ac.uk
¹ I.S.T.I “A.Faedo” ² Dept. of Computer Science ³ Dept. of Computer Science
CNR Pisa, Italy University of Pisa, Italy University of Leicester, UK

Abstract. We describe CAT, a toolkit supporting the analysis of communication-centric applications, i.e., applications consisting of ensembles of interacting services. Services are modelled in CAT as *contract automata* and communication safety is defined in terms of *agreement properties*. With the help of a simple (albeit non trivial) example, we demonstrate how CAT can (i) verify agreement properties, (ii) synthesise an orchestrator enforcing communication safety, (iii) detect misbehaving services, and (iv) check when the services form a choreography. The use of mixed-integer linear programming is a distinguished characteristic of CAT that allows us to verify context-sensitive properties of agreement.

1 Introduction

Communication is increasingly important in modern applications, especially for the distributed ones. Remote procedure/method invocations are loosing their prominence at the application level due to their poor scalability, crucial in modern distributed applications. Communication-based modelling is also appealing for non-distributed software. For instance, application-level protocols can be devised to specify the behavioural constraints ensuring the correct use of a library or of an off-the-shelf component. This trend is also witnessed by the growth that service-oriented or cloud computing had in the software industry. In this context, composition of software becomes paramount and requires proper theoretical foundations as well as tool support. In fact, although scalable, communication-centric applications may pose non trivial obstacles to validation.

We showcase CAT, a prototype toolkit supporting the validation of communication-centric applications. This toolkit (available at [3]) is based on *contract automata* [4,5,6], a recently proposed formal model of service composition. Contract automata abstractly describe (the communication pattern of) services as automata whose transitions represent *requests* and *offers*. An interaction between two services occurs when a *match* action is possible, that is when one service’s offer matches a partner’s request. Intuitively, contract automata capture the behaviour of services by tracking the interactions they are keen to execute with each other. Service composition is naturally described in terms of product automata. The matching between offers and requests has to guarantee *agreement* properties that amount to *safe* communications. Intuitively, an automaton

[★] The first three authors have been partially supported by project PRA_2016_64 “Through the fog” funded by the University of Pisa.

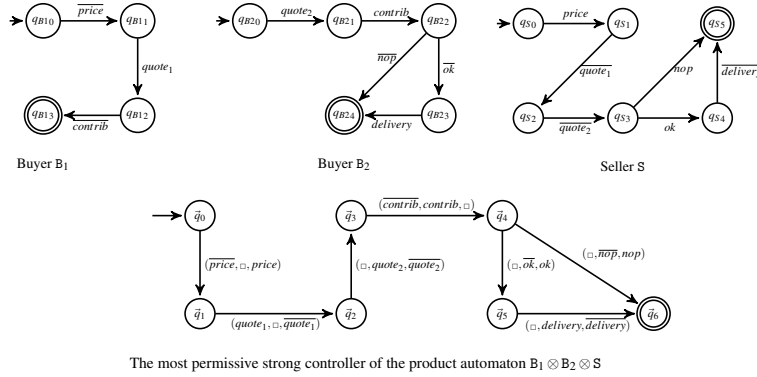


Fig. 1. The contract automata for 2BP

admits *strong agreement* if it has at least one trace made only by match transitions; and it is *strongly safe* if all the traces are in strong agreement. Basically, strong agreement guarantees that the composition of services has a sound execution, while strong safety guarantees that *all* executions of the composition are sound. Likewise for *agreement* but for the fact that traces also admit (unmatched) offers to model interactions with an external environment.

By means of an example we describe how the analysis of communication-centric applications can be supported by CAT. To this purpose we borrow here the two-buyers protocol (2BP) from [10] which we now briefly recall. Two buyers, say B_1 and B_2 , collaborate in purchasing an item from a seller S . Buyer B_1 starts the protocol by asking S the price of the desired item (*price*); the seller S makes an offer by sending the message *quote₁* to B_1 and the message *quote₂* to B_2 . Once received its quote, buyer B_1 sends to B_2 its contribution for purchasing the item (message *contrib*). Buyer B_2 waits for the quote from S and the contribution from B_1 . Then, it decides whether to terminate by issuing the *nop* message to S , or to proceed by sending an acknowledgement to S . In the latter case, S sends the item to B_2 (*delivery*), while if it receives *nop* it terminates with no further action. Figure 1 shows the contract automata of B_1 , B_2 , and S where each interaction is split in offers (over-lined labels) and requests (non over-lined labels).

We will apply CAT to the above protocol and show how, when the agreement property of interest is violated, we identify and fix defects.

2 Background: Contract Automata

Contract automata have been introduced in [4] from which we borrow the following definitions. Intuitively, a contract automaton represents the behaviour of a set of *principals* capable of performing some *actions*; more precisely, as formalised in Def. 1, the actions of contract automata allow them to “advertise” offers, “make” requests, or “handshake” on matching offer/request actions. The number of principals in a contract automaton is its *rank*, and a vectorial representation is used for tracking the moves of

each principal in it. The transitions are labelled with tuples in the set $\mathbb{L} \stackrel{\text{def}}{=} \mathbb{R} \cup \mathbb{O} \cup \{\square\}$ where: requests of principals will be built out of $\mathbb{R}$ while their *offers* will be built out of $\mathbb{O}$, $\mathbb{R} \cap \mathbb{O} = \emptyset$, and $\square \notin \mathbb{R} \cup \mathbb{O}$ is a distinguished label to represent components that stay idle. We let $a, b, c, \dots$ range over $\mathbb{L}$ and fix an involution $\bar{\cdot} : \mathbb{L} \rightarrow \mathbb{L}$ such that

$$\bar{\mathbb{R}} \subseteq \mathbb{O}, \quad \bar{\mathbb{O}} \subseteq \mathbb{R}, \quad \forall a \in \mathbb{R} \cup \mathbb{O} : \bar{\bar{a}} = a, \quad \text{and} \quad \bar{\square} = \square$$

Offer actions are overlined, e.g. $\bar{a}$. Let $\vec{v} = (a_1, \dots, a_n)$ be a vector of *rank* r_v , then $\vec{v}_{(i)}$ denotes the i -th element. We write $\vec{v}_1 \vec{v}_2 \dots \vec{v}_m$ for the concatenation of m vectors $\vec{v}_i$; $|\vec{v}| = n$ is the rank of $\vec{v}$; and $\vec{v}^n$ for the vector obtained by n concatenations of $\vec{v}$.

Definition 1. A tuple $\square^* b \square^*$ on $\mathbb{L}$ is a request (resp. offer) action on b iff $b \in \mathbb{R}$ (resp. $b \in \mathbb{O}$). A match (action) on b is a tuple $\square^* b \square^* \bar{b} \square^*$ on $\mathbb{L}$ ($b \in \mathbb{R} \cup \mathbb{O}$). Let $\bowtie \subseteq \mathbb{L}^* \times \mathbb{L}^*$ be the symmetric closure of $\bowtie \subseteq \mathbb{L}^* \times \mathbb{L}^*$ where $\vec{a}_1 \bowtie \vec{a}_2$ iff $|\vec{a}_1| = |\vec{a}_2|$ and both the following conditions hold

- $\exists b \in \mathbb{R} \cup \mathbb{O} : \vec{a}$ is either a request or an offer on b ;
- $\exists b \in \mathbb{R} \cup \mathbb{O} : \vec{a}_1$ is an offer on $b \implies \vec{a}_2$ is a request on b
- $\exists b \in \mathbb{R} \cup \mathbb{O} : \vec{a}_1$ is a request on $b \implies \vec{a}_2$ is a offer on b .

Definition 2. Assume as given a finite set of states $\mathfrak{Q} = \{q_1, q_2, \dots\}$. Then a contract automaton $\mathcal{A}$ of rank n is a tuple $\langle Q, \vec{q}_0, A^r, A^o, T, F \rangle$, where

- $Q = Q_1 \times \dots \times Q_n \subseteq \mathfrak{Q}^n$
- $\vec{q}_0 \in Q$ is the initial state
- $A^r \subseteq \mathbb{R}, A^o \subseteq \mathbb{O}$ are finite sets (of requests and offers, respectively)
- $F \subseteq Q$ is the set of final states
- $T \subseteq Q \times A \times Q$ is the set of transitions, where $A \subseteq (A^r \cup A^o \cup \{\square\})^n$ and if $(\vec{q}, \vec{a}, \vec{q}') \in T$ then both the following conditions hold:
 - $\vec{a}$ is either a request or an offer or a match
 - if $\vec{a}_{(i)} = \square$ then it must be $\vec{q}_{(i)} = \vec{q}'_{(i)}$

A principal is a contract automaton of rank 1 such that $A^r \cap co(A^o) = \emptyset$.

A principal is not allowed to make a request on actions that it offers. We have two different operators for composing contract automata, that interleave or match the transitions of their operands. We only force a synchronisation to happen when two contract automata are ready on their respective request/offer action. These operators represent two different policies of orchestration. The first operator, called *product*, considers the case when a service S joins a group of services already clustered as a single orchestrated service S' . In the product of S and S' , the first can only accept the still available offers (requests, respectively) of S' and vice versa. In other words, S cannot interact with the principals of the orchestration S' , but only with it as a whole component.

The second operation of composition, called *a-product*, puts instead all the principals of S at the same level of those of S' . Any matching request-offer of either contracts are split, and the offers and requests become available again, and are re-combined with complementary actions of S , and viceversa. The a-product turns out to satisfactorily model coordination policies in dynamically changing environments, because the

a-product is a form of *dynamic orchestration*, that adjusts the workflow of messages when new principals join the contract.

We now introduce our first operation of composition; recall that we implicitly assume the alphabet of a contract automaton of rank m to be $A \subseteq (A^r \cup A^o \cup \{\square\})^m$.

Definition 3 (Product). Let $\mathcal{A}_i = \langle Q_i, \vec{q}_{0i}, A_i^r, A_i^o, T_i, F_i \rangle, i \in 1 \dots n$ be contract automata of rank r_i . The product $\bigotimes_{i \in 1 \dots n} \mathcal{A}_i$ is the contract automaton $\langle Q, \vec{q}_0, A^r, A^o, T, F \rangle$ of rank $m = \sum_{i \in 1 \dots n} r_i$, where:

- $Q = Q_1 \times \dots \times Q_n$, where $\vec{q}_0 = \vec{q}_{01} \dots \vec{q}_{0n}$
- $A^r = \bigcup_{i \in 1 \dots n} A_i^r$, $A^o = \bigcup_{i \in 1 \dots n} A_i^o$
- $F = \{\vec{q}_1 \dots \vec{q}_n \mid \vec{q}_1 \dots \vec{q}_n \in Q, \vec{q}_i \in F_i, i \in 1 \dots n\}$
- T is the least subset of $Q \times A \times Q$ s.t. $(\vec{q}, \vec{c}, \vec{q}') \in T$ iff, when $\vec{q} = \vec{q}_1 \dots \vec{q}_n \in Q$,
either there are $1 \leq i < j \leq n$ s.t. $(\vec{q}_i, \vec{a}_i, \vec{q}'_i) \in T_i$, $(\vec{q}_j, \vec{a}_j, \vec{q}'_j) \in T_j$, $\vec{a}_i \bowtie \vec{a}_j$ and

$$\begin{cases} \vec{c} = \square^u \vec{a}_i \square^v \vec{a}_j \square^z \text{ with } u = r_1 + \dots + r_{i-1}, v = r_{i+1} + \dots + r_{j-1}, |\vec{c}| = m \\ \text{and} \\ \vec{q}' = \vec{q}_1 \dots \vec{q}_{i-1} \vec{q}'_i \vec{q}_{i+1} \dots \vec{q}_{j-1} \vec{q}'_j \vec{q}_{j+1} \dots \vec{q}_n \end{cases}$$
or there is $1 \leq i \leq n$ s.t. $(\vec{q}_i, \vec{a}_i, \vec{q}'_i) \in T_i$ and

$$\begin{aligned} \vec{c} &= \square^u \vec{a}_i \square^v \text{ with } u = r_1 + \dots + r_{i-1}, v = r_{i+1} + \dots + r_n, \text{ and} \\ \vec{q}' &= \vec{q}_1 \dots \vec{q}_{i-1} \vec{q}'_i \vec{q}_{i+1} \dots \vec{q}_n \text{ and} \\ &\forall j \neq i, 1 \leq j \leq n, (\vec{q}_j, \vec{a}_j, \vec{q}'_j) \in T_j \text{ it does not hold that } \vec{a}_i \bowtie \vec{a}_j. \end{aligned}$$

To retrieve the principals involved in a contract automaton obtained through the product introduced above, we introduce the following:

Definition 4 (Projection). Let $\mathcal{A} = \langle Q, \vec{q}_0, A^r, A^o, T, F \rangle$ be a contract automaton of rank n , then the projection on the i -th principal is $\Pi^i(\mathcal{A}) = \langle \Pi^i(Q), \vec{q}_{0(i)}, \Pi^i(A^r), \Pi^i(A^o), \Pi^i(T), \Pi^i(F) \rangle$ where $i \in 1 \dots n$ and:

$$\begin{aligned} \prod^i(Q) &= \{\vec{q}_{(i)} \mid \vec{q} \in Q\} & \prod^i(F) &= \{\vec{q}_{(i)} \mid \vec{q} \in F\} & \prod^i(A^r) &= \{a \mid a \in A^r, (q, a, q') \in \prod^i(T)\} \\ \prod^i(A^o) &= \{\vec{a} \mid \vec{a} \in A^o, (q, \vec{a}, q') \in \prod^i(T)\} & \prod^i(T) &= \{(\vec{q}_{(i)}, \vec{a}_{(i)}, \vec{q}'_{(i)}) \mid (\vec{q}, \vec{a}, \vec{q}') \in T \wedge \vec{a}_{(i)} \neq \square\} \end{aligned}$$

We now define the associative product.

Definition 5 (a-Product). Let $\mathcal{A}_1, \mathcal{A}_2$ be two contract automata of rank n and m , respectively, and let $I = \{\prod^i(\mathcal{A}_1) \mid 0 < i \leq n\} \cup \{\prod^j(\mathcal{A}_2) \mid 0 < j \leq m\}$. Then the a-product of $\mathcal{A}_1$ and $\mathcal{A}_2$ is $\mathcal{A}_1 \boxtimes \mathcal{A}_2 = \bigotimes_{\mathcal{A}_i \in I} \mathcal{A}_i$.

3 CAT at work

We have implemented CAT in Java according to the simple architecture of Figure 2. The main class of CAT extends JAMATA [3], a framework for manipulating automata yielding methods for loading, storing, printing, and representing finite state automata. In other words, CAT originally specializes JAMATA on contract automata, offering to the



Fig. 2. The architecture of CAT

developers an API for creating and verifying contract automata. Also, CAT interfaces with a separate module for solving linear optimization problems, called AMPL [8], described in Section 5. This is an original facet of CAT; in fact, it maps the (check of) agreement properties of interest on a linear optimization problem.

The user of CAT has access to its API, that can be conceptually classified as follows:

Automata operations consist of the methods `CA proj(int i)`, that returns the automaton specifying the i^{th} service of the composition, `CA product(CA[] aut)` and `CA aproduct(CA[] aut)` that compute respectively the product and the *as-associative* product of contract automata. Interestingly, `product` has to filter out the offers and request transitions when the source state has a corresponding outgoing match transition. Method `aproduct` is built on top of `product` by invoking `product` on the services obtained as projections of the automaton in input.

Safety check consists of the instance methods `safe`, `agreement`, `strongAgreement`, and `strongSafe` returning `true` if the corresponding agreement property holds on the contract automaton. Section 5 discusses the property of *weak agreement*.

Controllers consist of the methods `CA mpc()` and `CA smpc()` that return the *most permissive controller* (MPC), for respectively agreement and strong agreement. A controller basically represents the largest (strongly) safe sub-automaton and is obtained through a standard construction of Control Theory [7].

Liability detection consists of the methods `CATransition[] liable()` - returning transitions from a state s to a state t such that s is in the MPC but t is not - and `CATransition[] strongLiable()` that similarly returns such transitions for the MPC of the strong agreement property. In particular, liable services are those responsible for leading a contract composition into a failure.

Decentralization includes `int[][] branchingCondition()`, that returns two states and an action for which the branching condition is violated. Basically, the branching condition holds if the actions of a service are not affected by the states of the other services in the composition. Another similar method that deals with open-ended interactions is `int[][] extendedBranchingCondition()`. The last method in this category is `int[] mixedChoice()` that returns a mixed-choice state (a state where a principal has enabled both offers and requests inside matches). All such methods return `null` when the conditions they check do not hold.

We describe how to interact with the API of CAT through a simple command line interface (we plan to develop a GUI as well). The API is displayed and the user can choose one of the options (this is not shown here). Each displayed option corresponds to one of the methods described above. For instance, after choosing to compute a product, the user is asked to set the contract automata on which to take the product.

```

Output 1
-----
Do you want to create/load other contract automata? yes      1
Insert the name of the automaton to load or leave empty for create a new one: B1      2
                                                                    3
Contract automaton:                                          4
Rank: 1, Number of states: [4], Initial state: [0], Final states: [[3]]             5
Transitions: ([0],[1],[1]), ([1],[-2],[2]), ([2],[3],[3])    6
                                                                    7
Do you want to create/load other contract automata? yes      8
-----

```

The user inputs the automata in CAT by providing their file names (line 2 of Output 1) and `yes` on line 8 until there are no more automata to load (in which case the user enters `no` to obtain the result of the product). For each entered automaton, CAT prints a textual description on the screen (lines 4-6 in Output 1) reporting the rank, initial and final states, and the list of transitions. The transitions are triples (s, l, t) where s is the source state, l is the label, and t is the target state. These elements are lists of length r (the rank of the automaton), for instance, in Output 1 $r = 1$ (cf. on line 5). The i -th element of each list corresponds to the i -th service. In particular, the i -th action in the list of labels identifies the action performed by the i -th service; such action is strictly positive (if the action is an offer), strictly negative (if it is a request), and 0 if the service is idle in the transition. For B1, actions *price*, *quote*₁, and *contrib* are represented with the integers 1, -2, and 3, respectively.

First CAT computes the product automaton, and then it displays the result ($B1 \times B2 \times S$ in our example and stores it in a file named `B1xB2xS.data`). From the main menu, the user can now choose to compute the MPC of the product automaton (shown in Figure 1); the result is displayed in Output 2 below. Once the product automaton is loaded, CAT will compute the MPC:

```

Output 2
-----
The most permissive controller for strong agreement is:      1
Rank: 3                                                       2
Number of states: [4, 5, 6], Initial state: [0, 0, 0], Final states: [[3][4][5]]    3
Transitions:                                                  4
([0, 0, 0],[1, 0, -1],[1, 0, 1]) ([1, 0, 1],[-2, 0, 2],[2, 0, 2])             5
([2, 1, 3],[3, -3, 0],[3, 2, 3]) ([2, 0, 2],[0, -7, 7],[2, 1, 3])             6
([3, 2, 3],[0, 4, -4],[3, 3, 4]) ([3, 2, 3],[0, 5, -5],[3, 4, 5])             7
([3, 3, 4],[0, -6, 6],[3, 4, 5])                                     8
Do you want to save this automaton? (write yes or no) yes KS_B1xB2xS          9
-----

```

The resulting automaton is of rank 3 and corresponds to the MPC of Figure 1. The final states are represented as a list where the i -th element is the list of the final states of the i -th service. This representation allows to check if a state of the MPC is final or not without needing to explicitly enumerate all the final states of the MPC.

The transitions on lines 5-8 in Output 2 represent the transitions of the MPC; note that in each transition there is always an idle service. For instance, consider the transition $([0, 0, 0], [1, 0, -1], [1, 0, 1])$: it corresponds to the transition $(\vec{q}_0, (\overline{price}, \square, price), \vec{q}_1)$ of the MPC in Fig. 1 (the second component of the label is 0 because B2 is idle). The MPC can now be saved in a file as per line 9 in Output 2.

The underlying coordination mechanism of contract automata is orchestration. More precisely, services are oblivious of their partners and exchange messages through a “hidden” orchestrator (formalised by the MPC, if any). Whenever possible, one would like to have services interacting without the “supervision” of an orchestrator, using FIFO

buffers. Mild conditions [6] ensure that choreographies are sound, in other words that all the interactions among services are successful. We briefly discuss this issue below. For synchronous interactions (where buffers have size 1 and a single buffer may be non empty), services have to enjoy the *branching condition* that is necessary and sufficient for services to form a sound choreography. As said, a branching condition guarantees “unsupervised” communications soundness when the communication are synchronous [6,5]. However, such branching condition does not suffice for asynchronous interactions (namely when buffers are unbounded and more than one buffer is possibly non empty). In this case, an additional sufficient and commonly required condition is the absence of *mixed choice states*, i.e., states where more than one service can perform an offer (see [6]). Consider now Fig. 1, where in Output 3 the state $\vec{q}_2$ corresponds to $[2, 0, 2]$, the state $\vec{q}_3$ to $[2, 1, 3]$, and the transition $(\overline{contrib}, contrib, \square)$ to the label $[3, -3, 0]$. The MPC does not enjoy the branching condition, as CAT reports:

	Output 3	
State $[2,0,2]$ violates the branching condition because it has no transition labelled		1
$[3,-3,0]$ which is instead enabled in state $[2,1,3]$		2

It is important to observe that the message in Output 3 also flags states and transitions for which the condition is violated. We discuss the problem by considering the automata in Fig. 1. The local state of buyer B_1 in $\vec{q}_2$ and $\vec{q}_3$ is q_{B12} , while the locale state of B_2 in $\vec{q}_2$ is q_{B20} , and in $\vec{q}_3$ is q_{B21} . Therefore, in the case that B_2 is in local state q_{B20} where it is waiting for $\overline{quote}_2$, without an orchestrator the offer $\overline{contrib}$ from B_1 could fill up the 1-buffer of B_2 , leading to a deadlock. A simple fix consists in swapping the order in which the quotes are sent by the seller; CAT reports that the amended protocol (not shown here) enjoys the branching condition. The contract automaton has no mixed choice states, as detected by CAT. A mixed choice state could be introduced in 2BP if, e.g., B_2 could send the acknowledgement to S or receive $\overline{contrib}$ from B_1 in any order. For this variant of 2BP, CAT finds the mixed choice state, so showing that these services do not form a sound choreography.

4 Detailing the Implementation of CAT

CAT consists of a class `CAUtil` and of other classes `CA` and `CATransition`, extending two corresponding super-classes of JAMATA. The class `CA` provides the main functionalities of CAT; its instance variables capture the basic structure of our automata:

- `int rank` is the rank of the automaton;
- `int[] initial` is the initial state of the automaton (the array is of size `rank`);
- `int[] states` the vector of the number of local states of each principal in the contract automaton (the array is of size `rank`);
- `int[][] finalstates` the final states of each principal in the contract automaton;
- `CATransition[] tra` the transitions of the contract automaton.

The n local states of a principal are represented as integers in the range $0, \dots, n-1$; in this case, `states.length = 1` and `states[0] = n`. The state of an automaton of rank $m > 1$ is an m -vector `states` such that `states[i]` yields the number of states of

the i^{th} principal. This low-level representation (together with the encoding of actions and labels as integers) enabled us to optimize space.

The class `CATransition`, describes a transition of a contract automaton. The instance variables of a `CATransition` object are:

- `int[] source` (the starting state of the transition);
- `int[] label` (the label of the transition);
- `int[] target` (the arriving state of the transition).

The class `CATransition` provides methods to extract its instance variables, to check if the transition is an offer, a request or a match, and to extract the (index of the) principal performing the offer, if any.

5 Linear programming and contract automata

The properties of weak agreement were introduced for solving circularity issues, in which all services are stuck waiting the fulfilment of their requests before providing the corresponding offers [4]. For example, consider the services (rendered as regular expressions) $A = a.\bar{b}$ and $B = b.\bar{a}$; their product does not admit agreement. Circularity is solved by allowing matches between requests and offers even though they are not simultaneous; intuitively, offers may be fired “on credit” provided that the corresponding requests are honoured later on. A trace of an automaton is a weak agreement if for each request there is a corresponding offer, no matter in which order they occur in the trace. The notions of *admitting* weak agreement and of *weakly* safety are then similar to the ones of (strong) agreement reviewed earlier. For example, $A \otimes B$ admits weak agreement. The underlying theory and the decision procedures for the properties of weak agreement are developed in [4], and are formalised as mixed linear integer programming. This is because the properties of weak agreement are context-sensitive, and thus no controller can exist, i.e., a contract automaton for enforcing them. Below, we briefly review a component for solving the optimization problems related to contract automata, that complements the functionalities offered by CAT.

The decision procedures are implemented in *A Mathematical Programming Language* (AMPL) [8], a widely used language for describing and solving optimization problems. In this way, the automatic verification of contract automata under properties of weak agreement exploits efficient techniques and algorithms developed in the area of operational research. We now briefly describe the implementation of the techniques for verifying weak agreement. The script `flow.run`, to be launched with the command `ampl`, is described below:

```

----- flow.run -----
#reset;
option solver cplex;           // use the simplex algorithm in C
model weakagreement.mod;       // select model for weak agreement
data flow.dat;                 // load
solve;                          // apply the simplex algorithm
display gamma;                 // display the result: if gamma >= 0 then property holds

```

The script firstly loads the automaton from the file `flow.dat` (line 4). The description of the automata consists of the number of nodes, the cardinality of the alphabet of actions,

and a matrix of transitions for each action a , where there is value 0 at position (s, t) if there is no transition from state s to state t labelled by a , and respectively 1 or -1 if there is an offer or request transition on a . In this case, the contract automaton described in `flow.dat` is representative.

The AMPL linear program to load is given as input parameter to the script (line 3). The two optimization problems available are: `weakagreement.mod`, the file contains the formalization of the optimization problem for deciding whether a contract automaton admits weak agreement, and `weaksafety.mod` that contains the formalization of the optimization problem for deciding whether a contract automaton is weakly safe.

Both formal descriptions are then solved using the solver `cplex`, that is the simplex method implemented in C. However it is possible to select other available solvers in the script `flow.run` (line 2). The execution of the script will prompt to the user the value of variables. As proved in [4], if the variable `gamma` is non negative then the contract automaton satisfies the given property. Bi-level optimization problems can not be defined directly in AMPL. Therefore, we cannot plainly apply formalisation of [4] for representing weakly liable transitions as an optimization problem. However, different techniques of relaxation of the bi-level problem for over approximating the set of weakly liable transitions can be used, as for example lagrangian relaxation. As future work, we are planning to develop a toolchain for fully integrating the above techniques in CAT, in order to reuse them for the functionalities described in Section 3. In particular, CAT will automatically generate a contract automaton description `flow.dat`, execute the script `flow.run` and collect the results. The code of `weakagreement.mod` and `weaksafety.mod` is depicted in Figure 3. For further details about CAT, we refer the interested reader to the full documentation, available online at [3].

6 Concluding Remarks

We described CAT, a tool supporting the analysis of communication-centric applications attained with novel techniques based on combinatorial optimization. A non trivial example was used to show main features of CAT.

An interesting application domain for CAT are service-oriented applications. In this context, model-driven approaches have been advocated for the analysis of service composition. In particular, automata have been used as target models to translate BPEL processes [11] in [13,9]; for instance, constraint automata semantics of REO [1,2] is used in [12] to analyse web-services. Relations of contract automata with service composition are studied in [4,5,6]. The properties verified by CAT have not been considered by other approaches. For example, the identification - even in presence of circular dependencies of services (see Section 5) - of liable transitions that may spoil a composition complement the verification done in [12]. We conjecture that it would be possible to define model transformations from contract automata to BPEL which preserve the analysis discussed here.

A model-driven approach would also ease the integration of CAT with e.g., the tools discussed above. This would provide developers with a wide variety of tools for guaranteeing the quality of the composition of services according to different criteria.

```

weakagreement.mod
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32

# n number of nodes # m number of actions
param n; param m; param K; param final; #final node
set N := {1..n}; set M := {1..m}; param t{N,N}; param a{N,N,M};
var x_t{N,N} >=0 integer; var z_t{N,N,N} >=0;
var gamma; var p{N} binary; var wagreement;

#flow constraints
subject to Flow_Constraints {node in N}:
    sum{i in N}( x_t[i,node]*t[i,node] ) - sum{i in N}(x_t[node,i]*t[node,i]) =
        if (node == 1) then -1
        else if (node == final) then 1
        else 0;
;

subject to p1{node in N}: p[node] <= sum{i in N}( x_t[node,i]*t[node,i]);
subject to p2{node in N}: sum{i in N}( x_t[node,i]*t[node,i]) <= p[node]*K;

subject to Auxiliary_Flow_Constraints {snode in N diff {1},node in N}:
    sum{i in N}( z_t[snode,i,node]*t[i,node] ) - sum{i in N}(z_t[snode,node,i]*t[node,i]) =
        if (node == 1) then - p[snode]
        else if (node == snode) then p[snode]
        else 0;

subject to Auxiliary_Flow_Constraints2{i in N, j in N,snode in N}:
    z_t[snode,i,j]*t[i,j] <= x_t[i,j]*t[i,j];

subject to threshold_constraint {act in M}:
    sum{i in N,j in N} x_t[i,j]*t[i,j]*a[i,j,act] >= gamma;

#objective function
maximize cost: gamma;

```

```

weaksafety.mod
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34

# n number of nodes # m number of actions
param n; param m; param K; param final; #final node
set N := {1..n}; set M := {1..m}; param t{N,N}; param a{N,N,M};
var x_t{N,N} >=0 integer; var z_t{N,N,N} >=0;
var gamma; var p{N} binary; var v{M} binary; var wagreement;

#flow constraints
subject to Flow_Constraints {node in N}:
    sum{i in N}( x_t[i,node]*t[i,node] ) - sum{i in N}(x_t[node,i]*t[node,i]) =
        if (node == 1) then -1
        else if (node == final) then 1
        else 0;
;

subject to p1{node in N}: p[node] <= sum{i in N}( x_t[node,i]*t[node,i]);
subject to p2{node in N}: sum{i in N}( x_t[node,i]*t[node,i]) <= p[node]*K;

subject to Auxiliary_Flow_Constraints {snode in N diff {1},node in N}:
    sum{i in N}( z_t[snode,i,node]*t[i,node] ) - sum{i in N}(z_t[snode,node,i]*t[node,i]) =
        if (node == 1) then - p[snode]
        else if (node == snode) then p[snode]
        else 0;

subject to Auxiliary_Flow_Constraints2{i in N, j in N,snode in N}:
    z_t[snode,i,j]*t[i,j] <= x_t[i,j]*t[i,j];

subject to vi: sum{i in M} v[i] = 1;

subject to threshold_constraint :
    sum{act in M,i in N,j in N} (v[act]*x_t[i,j]*t[i,j]*a[i,j,act]) <= gamma;

#objective function
minimize cost: gamma;

```

Fig. 3. The implementation in AMPL of the optimization problem for deciding weakagreement and weak safety.

The tool is still a prototype; we plan to improve its efficiency, extend it with new functionalities (e.g., relaxation), and improve its usability (e.g., adding a user-friendly GUI and pretty-printing automata). We note that CAT provides a valid support to the analysis of applications. In fact, CAT is able to detect possible violations of the properties of interest (for example branching condition, mixed choice). A drawback of CAT is that it does not support modelling and design of applications. An interesting evolution of CAT would be to add functionalities for amending applications violating properties of interest. For instance, once liable transitions are identified, CAT could suggest how to modify services to guarantee the property. This may also be coupled with the model-driven approach by featuring functionalities tracing transitions in the actual source-code of services.

References

1. F. Arbab. Reo: a channel-based coordination model for component composition. *Mathematical Structures in Computer Science*, 14(3):329–366, 2004.
2. C. Baier, M. Sirjani, F. Arbab, and J. J. M. M. Rutten. Modeling component connectors in reo by constraint automata. *Sci. Comput. Program.*, 61(2):75–113, 2006.
3. D. Basile. JAMATA and CAT. <https://github.com/davidebasile/workspace>.
4. D. Basile, P. Degano, and G. L. Ferrari. Automata for analysing service contracts. In M. Maffei and E. Tuosto, editors, *TGC 2014*, volume 8902 of *LNCS*, pages 34–50. Springer, 2014.
5. D. Basile, P. Degano, G. L. Ferrari, and E. Tuosto. From orchestration to choreography through contract automata. In *ICE 2014*, pages 67–85, 2014.
6. D. Basile, P. Degano, G. L. Ferrari, and E. Tuosto. Relating two automata-based models of orchestration and choreography. *Journal of Logical and Algebraic Methods in Programming*, 85(3):425 – 446, 2016.
7. C. G. Cassandras and S. Lafortune. *Introduction to Discrete Event Systems*. Springer-Verlag New York, Inc., Secaucus, NJ, USA, 2006.
8. R. Fourer, D. M. Gay, and B. W. Kernighan. *AMPL: A mathematical programming language*. AT&T Bell Laboratories Murray Hill, NJ 07974, 1987.
9. X. Fu, T. Bultan, and J. Su. Analysis of Interacting BPEL Web Services. In *WWW '04*, pages 621–630. ACM, 2004.
10. K. Honda, N. Yoshida, and M. Carbone. Multiparty asynchronous session types. In G. C. Necula and P. Wadler, editors, *POPL*, pages 273–284. ACM, 2008.
11. M. B. Juric. *Business Process Execution Language for Web Services BPEL and BPEL4WS 2Nd Edition*. Packt Publishing, 2006.
12. S. Tasharofi, M. Vakilian, R. Z. Moghaddam, and M. Sirjani. Modeling Web Service Interactions Using the Coordination Language Reo. In *WS-FM*, volume 4937 of *LNCS*. Springer, 2008.
13. A. Wombacher, P. Fankhauser, and E. Neuhold. Transforming BPEL into annotated deterministic finite state automata for service discovery. In *Web Services, 2004*, 2004.